

Furthering GUI-Grounding for Agentic AI: An Evolving Training Dataset

A Thesis submitted to the faculty of

San Francisco State University

In partial satisfaction of the

requirements for

the Degree

Masters of Science

in

Computer Science

by

André Ndaye Shannon

San Francisco, California

May 2026

Copyright by

André Ndaye Shannon

2026

Abstract

Graphical User Interface (GUI) grounding is key component enabling agentic AI to map natural-language instructions to specific screen elements. Recent work by Yuan et al. [10] proposed a self-evolutionary reinforcement learning method that iteratively filters the training dataset throughout training and their model, SE-GUI, achieves a notable performance on the ScreenSpot-Pro benchmark using both a small model and dataset. We build upon this self-evolutionary method, modifying the accumulation of attention maps used in the filtering method and testing on a lower quality dataset (SE-GUI dataset was not available). Our modified self-evolutionary method outperforms the original method on the lower quality dataset while approaching the SE-GUI reported results using far fewer training resources. This work highlights the importance of data quality in reinforcement learning and the potential of the modified self-evolutionary filtering method given more resources.

Acknowledgments

I would like to thank Dr. Isabel Song for her support and guidance even throughout her sabbatical. Thanks to Jacob Waters for the introduction to Dr. Song and the precursor project to this one. Thanks to San Francisco State University’s High-Performance Computing cluster for enabling me to conduct this research and the other users putting up with sharing the gpu cluster during my many training runs. Also thanks to ChatGPT and Claude for their coding assistance.

Table of Contents

Table of Contents	v
List of Tables	vi
List of Figures	vii
1 Introduction	1
2 Related Work	2
2.1 SE-GUI	2
2.2 GTA1	7
3 Method	9
3.1 Modifying Self-Evolutionary Filtering Method	9
3.2 Self-Evolution Filtering For Data Curation	11
3.3 Computation Environment	12
4 Experiment	13
4.1 Experimental Results for Modified Self-Evolution	13
4.2 Experimental Results for Data Curation	15
5 Conclusion and Future Work	18
5.1 Conclusion	18
5.2 Future Work	18
Bibliography	21

List of Tables

3.1	Difference in Hyperparameters Between SE-GUI Experiment and Ours.	12
4.1	Results for Modified Self-Evolution Filtering Experiment	14
4.2	Results of Curation Experiment with 3k Data	16
4.3	Results of Curation Experiment with 10k Data	16
4.4	Results of Curation with Self-Evolution Experiment	17

List of Figures

2.1	SE-GUI Aggregation of Attention Values	4
2.2	Example tool call output of model	7
3.1	Modified Aggregation of Attention Values	11

Introduction

Agentic Artificial Intelligence (AI) is a relatively new field that aims to leverage Large Language Models (LLMs) to create autonomous agents that can navigate a computer and complete complex multi-step tasks. The main goal is to have a system that a user can specify a desired goal (e.g. find the three shortest and cheapest flights from SFO to ATL) and have the agent complete the task - navigating searches, websites, and reasoning - with minimal to no input from the user.

Graphical User Interface (GUI) grounding is a subfield of agentic AI involving mapping natural-language instruction to specific visual elements on a screen (e.g. buttons, text fields) to perform actions like clicking or typing. This project focuses on the GUI grounding subfield and correctly identifying elements to left-click based on a short instruction.

Related Work

This project largely builds upon two key papers, Enhancing Visual Grounding for GUI Agents via Self-Evolutionary Reinforcement Learning (SE-GUI) [10] and GTA1: GUI Test-time Scaling Agent (GTA1) [8]. Broadly, the training data from GTA1 is used with modified training techniques of SE-GUI.

2.1 SE-GUI

The key contributions of SE-GUI [10] include techniques used for curating their 3k dataset, the dense policy gradient they defined for their reward function, and their self-evolutionary reinforcement finetuning mechanism that iteratively refines their training data using attention maps. They achieved state of the art results at the time for their 3B and 7B parameter models with only 3k of training data (orders of magnitude less training data than used by similar models). This project explores and builds upon their self-evolutionary mechanisms detailed in the next section.

Self-Evolution

Yuan et al. split the training of their model into stages where each stage trains for 10 epochs on their self-evolving training dataset for a total of 4 stages. For the first stage, the model is trained on their full initial dataset of 3k for 10 epochs. For each stage thereafter, the previous stage’s trained model is used to filter the original dataset by building an attention map for each sample and seeing if it is correctly focusing on the visual tokens of the user interface (UI) element corresponding to the instruction. The model is then trained for another 10 epochs on that filtered dataset. The intuition behind the self-evolution is that if the model is attending to the sample image appropriately, then it can learn from that and it should be included in the current training set. If the model is not attending well to the sample, then training on the sample is more likely to update the model parameters in the wrong direction.

Building the attention map for each sample involves running each sample through the previous stage’s trained model (teacher model), generating output tokens, and recording transformer attention values. This will yield attention vectors for every layer in the model and for every token - image tokens, instruction tokens, and generated tokens. Yuan et al. focus on the attention the generated tokens place on the image tokens, discarding all other attention values. The attention vectors for each generated token transformer layer is truncated to just values corresponding to image tokens and normalized. All layers are then averaged together and we assume an average of tokens as well, although this is not explicitly

Figure 2.1: SE-GUI Aggregation of Attention Values

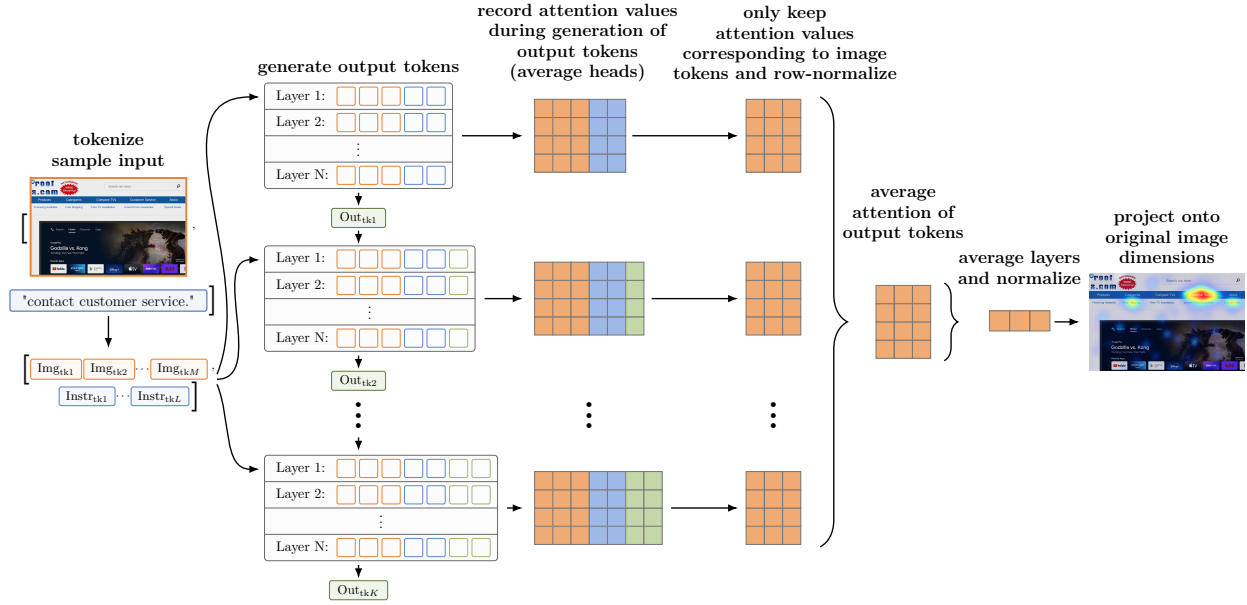


Figure 2.1: SE-GUI self-evolution, aggregation of attention values of generated tokens to the image tokens into one attention map.

stated in Yuan et al.'s paper. This aggregated attention weight vector is then projected onto the original image dimensions producing a spatial attention map highlighting the areas of the image that the model focuses on while generating the output. This aggregation process into the final attention map is shown in Fig. 2.1

With the attention map, Yuan et al. applies two filter methods, P_{peak} and P_{global} . The sample passes P_{peak} if there exists an attention value within the bounding box of the desired element that exceeds the threshold $\tau = 0.2$. To make this threshold reasonable, we assume

a max normalization on the final attention map so that every attention value is between 0 and 1 with the largest attention value being 1 and all other attention values some fraction of that. A sample passes P_{global} if the average attention value within the bounding box of the desired element is greater than the average attention value of the entire image. Since this is a relative comparison, how it is normalized doesn't make a difference, so we simply use an L1-normalization on the final attention map. A sample is added to the new evolved training dataset for that stage only if it passes both P_{peak} and P_{global} .

In Yuan et al.'s paper, they integrated this filtering into the reward function for training the model, multiplying the reward by 0 if the sample failed or by 1 if it passed. However, since the teacher model and initial dataset does not change for a given stage, the evolved training dataset for that stage can be fully precomputed and we present it this way for ease of understanding and because this is how we implemented it when recreating the SE-GUI training method.

Limitation of SE-GUI Self-Evolution

The way the attention values are aggregated into one attention map loses some important information. Before averaging layers and tokens, each layer is truncated to just values corresponding to visual tokens and is normalized. This means that information on layers that have significantly more or less attention to the image tokens as opposed to the instruction and generated tokens is lost. Layers that barely pay attention to image tokens get their attention

values artificially inflated while layers that pay most of their attention to the image tokens don't get a bigger say in the final attention map. There seems to be room for improvement on representing how much attention the model pays to the image tokens while generating output tokens.

Considering the difficulty in tracing the reasoning of transformer models, we don't know where in the model that the reasoning about the image and where to click happens. Each model layer is seen as a further abstraction of the previous layers and we don't have a way of telling which layer the model connects the natural language of the instruction to the element within the image. Considering this, it seems more appropriate that a sample should pass P_{peak} if any layer has a significant activation within the bounding box of the element and an average of layers would obfuscate this.

The self-evolution filtering looks at the attention of all the generated tokens to the image tokens. The generated output is a tool call that looks like Figure 2.2: Given that most of the of the output is tool call jargon, most of the output tokens probably are not focusing all that much on the image tokens except when generating the coordinates. It seems like the scope of the generated tokens used for attention values should be reduce to just those that would need information from the image.

Unfortunately, Yuan et al. have yet to publish their curated dataset and the code for the self-evolutionary part of their training. While not a limitation of their theoretical contributions, it makes it very difficult reproduce practically.

Figure 2.2: Example tool call output of model

```
<tool_call>
{"name": "computer_use", "arguments": {"action": "left_click",
  "coordinate": [3206, 137]}}
</tool_call>
```

2.2 GTA1

From the GTA1 work [8], we use their published training data, so this section will just give an overview of their dataset and not of their training methods and other contributions. Yang et al. use a conglomeration of datasets from several different sources: Aria-UI-Web [9], OmniACT [1], UI Vision [5], Widget Caption [3], and OS-Atlas-Desktop [7]. Each data sample consists of a screenshot, bounding box coordinates of the desired UI element, a human instruction, and coordinates for the ideal model response of where to left click.

Yang et al. found that some of the bounding boxes of the dataset were misaligned with the screenshot due to various rendering issues and applied a cleaning strategy. For each screenshot, they applied OmniParser [4], a Machine Learning (ML) tool for parsing elements from a screenshots, to get a list of UI elements' bounding boxes. They then take the maximum area of their intersection divided by the area of their union (Intersection over Union) between the given bounding box from the dataset and any of the OmniParser generated bounding boxes. If the maximum Intersection over Union is not greater than their

predefined threshold, then that sample is discarded.

The GTA1 dataset contains 1,559,535 total samples, with 1,050,911 desktop samples and 508,624 mobile samples. Considering that SE-GUI’s dataset started at 300k and was curated to just 3k, the GTA1 dataset is likely a lower quality than the SE-GUI’s dataset.

Method

The initial goal of this project was to improve GUI grounding in agentic AI. Inspired by SE-GUI and the need for high quality data when training using Reinforcement Learning (RL), we look at 1) improving upon SE-GUI’s self-evolutionary method with modifications to the filtering method and 2) using the data filtering for the Self-Evolutionary process for data curation.

3.1 Modifying Self-Evolutionary Filtering Method

Addressing the limitations of SE-GUI’s self-evolution from earlier, we would like P_{peak} and P_{global} to keep layer differences in their total attention to the image tokens during aggregation into one attention map. We also want P_{peak} , which looks for a significant attention value within the bounding box, to pass if any model layer has that significant attention as we don’t where in the model the abstraction and reasoning of the image and where to click actually happens. Lastly, we want to just look at the attention values corresponding to output tokens that would actually need information from the image and ignore output tokens that are tool call jargon.

To keep layer differences of total attention to the image tokens during aggregation, we drop the normalization after truncating each generated token transformer layer to just the image tokens and add attention values of different tokens together instead of averaging. For P_{peak} since we want to see if any layer passes, we max-normalize each layer and test each layer if it contains an attention value greater than $\tau = 0.2$ within the bounding box of the desired element. For P_{global} , we sum across layers as well and L1-normalize before testing if the average attention value within the bounding box is greater than the average attention value of the entire image. This aggregation for P_{peak} and P_{global} can be seen in Fig. 3.1. Summing across tokens and layers instead of normalizing and averaging will keep the contributions of each layer relative to the total attention they give to the image. This way, highly attentive layers to the image tokens will not be underrepresented and poorly attentive layers to the image tokens are not artificially inflated. Lastly, we only use the attention values of the output tokens corresponding to the predicted coordinates for the left click, ignoring the other output token attention values.

To test the modified self-evolutionary filtering method, we randomly sample 3.2k desktop samples from GTA1’s dataset (they also have mobile samples). We train three models 1) a baseline using SE-GUI’s training method (using their rewards function and RL training strategy) but without the self-evolution 2) a replication of SE-GUI with self-evolution and 3) SE-GUI with the modified self-evolution filtering method. At the end of each stage, the trained models are evaluated on the ScreenSpot-Pro benchmark [2]. There are a few differences in training hyperparameters than published in the SE-GUI paper, and we discuss

Figure 3.1: Modified Aggregation of Attention Values

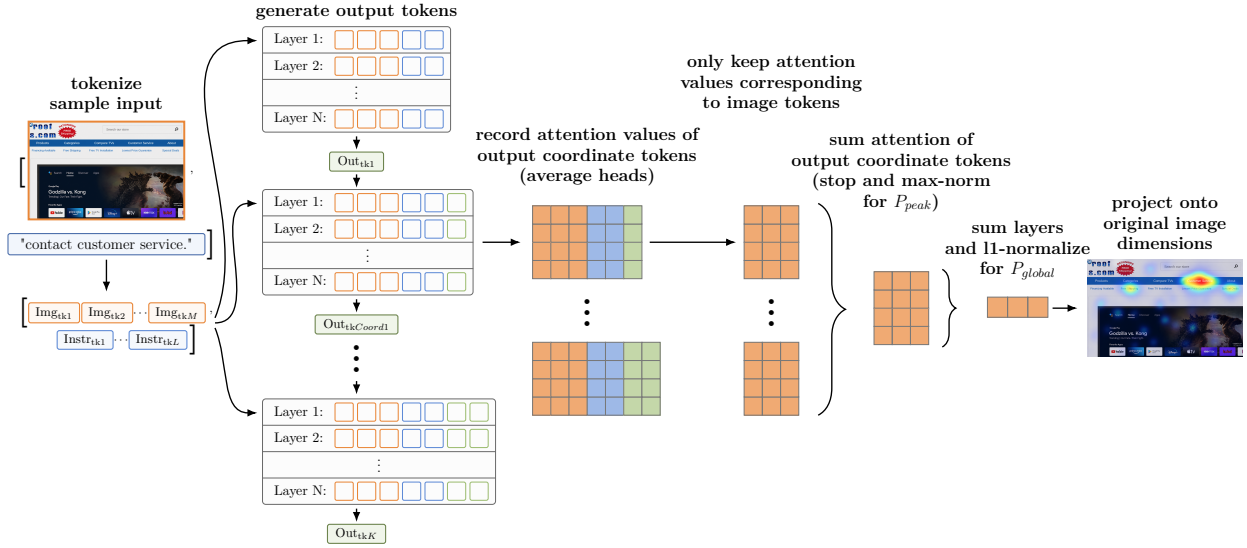


Figure 3.1: Modified SE-GUI self-evolution, aggregation of attention values of generated coordinate tokens to the image tokens.

those in Section 3.3.

3.2 Self-Evolution Filtering For Data Curation

Seeing the effectiveness of the self-evolution filtering in stage 2 and onward for training, what if it was applied to stage 1 as well? Filtering the data before stage 1 is equivalent to data curation, so would this make an effective data curation method? Without a trained model from the previous stage, we use the base Qwen2.5-VL-3B-Instruct model for obtaining the attention maps for filtering. The same 3.2k random desktop data from GTA1, is filtered

Table 3.1: Difference in Hyperparameters Between SE-GUI Experiment and Ours.

	SE-GUI	Our Experiment
# epochs per stage	10	4
# generations per sample	8	4
gradient accumulation steps	2	4
batch size per optimizer step	16	16

using the base Qwen2.5-VL-3B-Instruct model and we train models using 1) SE-GUI method without self-evolution to test just data curation and 2) SE-GUI method with the modified self-evolution filtering method (filtering from the 3.2K each stage).

3.3 Computation Environment

These experiments were run on the San Francisco State University (SFSU) High-Performance Computing (HPC) cluster which hosts 4 NVIDIA A100 (80GB) GPUs. The SE-GUI authors used 8 NVIDIA A100 (80GB) GPUs so there were a few hyperparameters that needed to be adjusted from SE-GUI training to work with the fewer resources as summarized in Table 3.1. Each stage was only run for 4 epochs instead of 10 to make training time more manageable. Each A100 GPU (80GB) was able to run one generation if the model was sharded across all 4 GPUs. This gave an upper bound on the number of generations per sample that could be done for the Group Relative Policy Optimization (GRPO) training [6]. To keep batch size per optimization step the same as SE-GUI, we increase the gradient accumulation steps from 2 to 4. This is intended to keep the training stable given the fewer generations per sample.

Experiment

4.1 Experimental Results for Modified Self-Evolution

The results for the modified self-evolutionary filtering method experiment are in Table 4.1. The training dataset size changes every stage with self-evolution and is noted under Train size. The Perf. column is the end of the stage’s trained model’s performance on the ScreenSpot-Pro benchmark. The SE-GUI reported results were numbers taken from their paper [10] for the first three stages, but they didn’t report how many samples were filtered out at each stage. The Baseline: no self-evolve is trained using SE-GUI’s method without the self-evolution, so the training is the same for all stages, just tested on the ScreenSpot-Pro benchmark at the end of every stage. For the SE-GUI replication, the self-evolutionary filtered dataset size seemed very small with P_{peak} being the main bottleneck, so we also tried filtering with just P_{global} .

We see that SE-GUI’s reported results from their paper achieve the highest score on the ScreenSpot-Pro benchmark. This is likely due to their 3k dataset being much more selectively curated than GTA1’s much larger dataset with over 1 million desktop samples. In addition, each stage trained for 10 epochs compared to our 4 epochs per stage and had 8

Table 4.1: Results for Modified Self-Evolution Filtering Experiment

Experiment	Stage 1		Stage 2		Stage 3	
	Train size	Perf.	Train size	Perf.	Train size	Perf.
SE-GUI reported results	3,018	31.90%	???	33.00%	???	35.70%
Baseline: no self-evolve	3200	29.79%	3200	31.05%	3200	31.69%
SE-GUI replication	3200	29.79%	265	30.87%	268	31.25%
SE-GUI replication (P_{global} only)	3200	29.79%	1235	31.06%	1231	31.82%
Self-evolve modified filtering	3200	29.79%	2941	33.71%	2945	34.41%

Table 4.1: Results for modified self-evolutionary filtering method for three stages. The training set size changes every stage and is noted under Train size and Perf. is each model’s accuracy on the ScreenSpot-Pro benchmark.

generations per sample instead of 4 in their GRPO steps, essentially 5 times more training. Given these differences, it is not a fair comparison but we choose to include it so readers had an idea of what the original SE-GUI results looked like for their 3k model.

The SE-GUI replication did not perform better than the baseline raising a few red flags. Yuan et al. did not publish a baseline of training without the self-evolution part of their method. Thus we are unsure what the expected performance should have been, but it presumably should have surpassed the baseline. We think this is due to GTA1 have lower quality data than SE-GUI’s highly curated 3k dataset and that their self-evolutionary method might necessitate having a high quality base dataset. However, seeing that the replicated SE-GUI achieved roughly the same performance on the benchmark as the baseline (within 0.5% at each stage) with much less training data, shows the possibility of using the SE-GUI

training method for reducing training time. This works as long as the time for filtering is less than what would be the training time for all the filtered out samples which is certainly the case with SE-GUI replication using both P_{peak} and P_{global} .

The modified self-evolutionary filtering method outperformed the baseline and SE-GUI replications, showing promise. This opens the door on applying this method to much bigger already existing data sets such as GTA1’s dataset without having to go through SE-GUI’s rigid curation. Interestingly, only about 8% of the original data is filtered out at each stage, keeping the majority of the dataset for training. This demonstrates that a small portion of the data samples can significantly hamper training and places more emphasis on data quality being a key factor in reinforcement learning.

4.2 Experimental Results for Data Curation

The results for using the modified self-evolutionary filtering method for data curation are presented in Table 4.2. We also test four epochs with a 10k subset of GTA1 data with results shown in Table 4.3. Since there is no self-evolution, there is no need for stages, but we still benchmark every four epochs to better compare the data. We see a significant jump in performance with the curated dataset for the first four epochs in both the 3.2k and 10k trainings, but epoch 8 and 12 of the 3.2k training both show performance dropping back to that of the baseline.

Using the modified self-evolutionary filtering method for curation seems to benefit in

Table 4.2: Results of Curation Experiment with 3k Data

Experiment	Epoch 4		Epoch 8		Epoch 12	
	Train size	Perf.	Train size	Perf.	Train size	Perf.
Baseline: no self-evolve	3200	29.79%	3200	31.05%	3200	31.69%
Currated: no self-evolve	2487	32.01%	2487	30.80%	2487	31.50%

Table 4.2: Results of using the modified self-evolutionary filtering for data curation with baseline for reference.

Table 4.3: Results of Curation Experiment with 10k Data

Experiment	Epoch 4	
	Train size	Perf.
Baseline: no self-evolve 10k	10000	30.61%
Currated: no self-evolve 10k	7770	33.52%

Table 4.3: Results of using the modified self-evolutionary filtering for data curation on a 10k subset of GTA1 data.

the short term for the first several epochs, but not have any long term benefits. This is likely due to the self-evolutionary filtering being geared towards filtering for samples the model is currently capable of learning (attending to the correct area will cause parameters associated with that area to be updated during training), and not all quality samples for GUI grounding. As the model gets better, we expect its attention to also get better and to be able to pass more samples during the next filtering, hence the evolution. However, using

Table 4.4: Results of Curation with Self-Evolution Experiment

Experiment	Stage 1		Stage 2		Stage 3	
	Train size	Perf.	Train size	Perf.	Train size	Perf.
SE-GUI reported results	3,018	31.90%	???	33.00%	???	35.70%
Baseline: no self-evolve	3200	29.79%	3200	31.05%	3200	31.69%
Self-evolve modified filtering	3200	29.79%	2941	33.71%	2945	34.41%
Currated: no self-evolve	2487	32.01%	2487	30.80%	2487	31.50%
Curation + SE modified filtering	2487	32.01%	2922	32.32%	2931	32.26%

Table 4.4: Results for the data curation with self-evolution in context of the previous results.

the self-evolutionary filtering for just data curation only looks at what the initial base model attended well to and does not evolve as the model gets better. Thus it never sees potentially good samples that it initially attended poorly to.

The next logical step would be to combine the curation with the self-evolution and we present these results in Table 4.4. Curiously, we do not see as big an improvement in performance at the later stages as with using just the modified self-evolution. This is somewhat baffling and raises some questions that we briefly explore in Section 5.2.

Conclusion and Future Work

5.1 Conclusion

The modified self-evolution shows potential in outperforming the baseline and SE-GUI replication on a lower quality dataset while approaching the SE-GUI reported results using far fewer resources. The self-evolution method boosts performance on the ScreenSpot-Pro benchmark by limiting the training samples that the models sees during training stages, showing the importance of data quality in improving model accuracy and training time. Using the self-evolution filtering for data curation seems to only work well for the earlier stages with later stage drawbacks, even when combined with the self-evolutionary training.

5.2 Future Work

While we were able to answer some of our initial questions, this project has left us with several more questions to explore. With more resources/time, we would like to see how this modified self-evolution method scales with data. This is readily doable, given the computational resources, as the experiments mostly worked with a 3.2k subset of GTA1's

data which had over 1 million desktop samples. Given access to SE-GUI’s training data, a more faithful comparison between their original self-evolution training and our modified one would be enlightening.

We would like to explore other data curation techniques alongside of the self-evolution filtering. GTA1’s curation involved leveraging OmniParser to check data quality at scale and got us thinking about other ways of using vision language models (VLMs) to judge the quality of samples at scale. Preliminary exploration of this with visual inspection of the samples showed promise. Many GTA1 samples had elements that did not fully render or instructions with multiple possible elements as solutions. A combination of the VLM judge and self-evolutionary filtering showed promise for filtering out many of these ambiguous samples.

A big unanswered question from this project was why the curation with the modified self-evolutionary filtering did not have similar later stage performance growth as with just using the modified self-evolutionary filtering method? Is higher quality data more important towards the end of training? Does maximizing quantity of data matter more for the earlier epochs and maximizing quality matter more for the later epochs? Models such as ChatGPT are pretrained on massive amounts of data at first and then go through supervised fine-tuning (SFT) and reinforcement learning from human feedback (RLHF) emphasizing the dataset size for pretraining and dataset quality with SFT and RLHF.

Some other areas this project piqued our interest in was using the filtering method to reduce training time while maintaining performance. Training takes a lot of computational and

time resources and this could help in resource constrained environments. Another questions we didn't explore was what is the ideal number of epochs per stage?

Bibliography

- [1] Raghav Kapoor et al. *OmniACT: A Dataset and Benchmark for Enabling Multi-modal Generalist Autonomous Agents for Desktop and Web*. 2024. arXiv: 2402.17553 [cs.AI]. URL: <https://arxiv.org/abs/2402.17553>.
- [2] Kaixin Li et al. *ScreenSpot-Pro: GUI Grounding for Professional High-Resolution Computer Use*. 2025. arXiv: 2504.07981 [cs.CV]. URL: <https://arxiv.org/abs/2504.07981>.
- [3] Yang Li et al. *Widget Captioning: Generating Natural Language Description for Mobile User Interface Elements*. 2020. arXiv: 2010.04295 [cs.LG]. URL: <https://arxiv.org/abs/2010.04295>.
- [4] Yadong Lu et al. *OmniParser for Pure Vision Based GUI Agent*. 2024. arXiv: 2408.00203 [cs.CV]. URL: <https://arxiv.org/abs/2408.00203>.
- [5] Shravan Nayak et al. *UI-Vision: A Desktop-centric GUI Benchmark for Visual Perception and Interaction*. 2025. arXiv: 2503.15661 [cs.CV]. URL: <https://arxiv.org/abs/2503.15661>.
- [6] Zhihong Shao et al. *DeepSeekMath: Pushing the Limits of Mathematical Reasoning in Open Language Models*. 2024. arXiv: 2402.03300 [cs.CL]. URL: <https://arxiv.org/abs/2402.03300>.

- [7] Zhiyong Wu et al. *OS-ATLAS: A Foundation Action Model for Generalist GUI Agents*. 2024. arXiv: 2410.23218 [cs.CL]. URL: <https://arxiv.org/abs/2410.23218>.
- [8] Yan Yang et al. *GTA1: GUI Test-time Scaling Agent*. 2025. arXiv: 2507.05791 [cs.AI]. URL: <https://arxiv.org/abs/2507.05791>.
- [9] Yuhao Yang et al. *Aria-UI: Visual Grounding for GUI Instructions*. 2025. arXiv: 2412.16256 [cs.HC]. URL: <https://arxiv.org/abs/2412.16256>.
- [10] Xinbin Yuan et al. *Enhancing Visual Grounding for GUI Agents via Self-Evolutionary Reinforcement Learning*. 2025. arXiv: 2505.12370 [cs.AI]. URL: <https://arxiv.org/abs/2505.12370>.